난해한 언어 모델

Subham Sekhar Sahoo*[1] Zhihan Yang*[2] Yash Akhauri†[1] Johnna Liu†[1] Deepansha Singh†[1] Zhoujun Cheng†[3] Zhengzhong Liu[3] Eric Xing[3] John Thickstun[2] Arash Vahdat[4] *공동 제1 저자 †공동 제2 저자 [1] 코넬 테크 [2] 코넬 대학교 [3]MBZUAI [4] 엔비디아 {ssahoo, zhihany}@cs.cornell.edu

초록

확산 기반 언어 모델은 병렬 및 제어 가능한 생성을 가능하게 함으로써 자기회귀(AR) 모델에 대한 매력적인 대안을 제시합니다. 이러한 모델군 중에서 마스크 확산 모델(MDM)은 가장 강력한 성능을 달성하지만, perplexity 면에서는 여전히 AR 모델에 미치지 못하며, 특히 KV 캐싱과 같은 핵심적인 추론 시간 효율성 기능이 부족합니다. 본 연구에서는 AR 및 MDM 패러다임을 융합하여 perplexity 사이의 부드러운 보간을 가능하게 하고 각 한계를 극복하는 새로운 모델군인 Eso-LM을 소개합니다. Eso-LM은 표준 언어 모델링 벤치마크에서 새로운 최첨단 성능을 확립합니다. 중요하게도, 우리는 병렬 생성을 유지하면서 MDM에 KV 캐싱을 도입한 최초의 사례이며, 이는 추론 효율성을 크게 향상시킵니다. 최적화된 샘플링 스케줄과 결합된 우리의 방법은 표준 MDM보다 최대 65배, 이전의 준자기회귀 접근 방식보다 4배 빠른 추론을 달성합니다. 코드와 모델 체크포인트는 프로젝트 페이지에서 제공됩니다.

https://s-sahoo.com/Eso-LMs

1 서론

우리는 언어 모델링의 중요한 전환점에 서 있습니다. 오랫동안 황금 표준으로 여겨져 온 자기회귀(AR) 언어 모델이 이제 표준 언어 생성 분야에서 확산 언어 모델과 경쟁하는 패러다임 전환이 진행 중입니다. 최근 연구 *[26, 29]*에 따르면 마스크 확산 모델(MDM)은 소규모 언어 벤치마크에서 AR 모델과의 격차를 좁히고 있으며, 분자 생성 [28, 11] 및 그래프 생성 *

이러한 결과는 MDM이 AR 모델에 대한 매력적인 대안이 되게 합니다. 그러나 MDM은 두 가지 주요 한계에 직면합니다. (1) 추론 속도: 병렬 생성을 지원함에도 불구하고, MDM은 실제로는 AR 모델보다 현저히 느린데, 이는 주로 채팅 시스템과 같은 실시간 애플리케이션에 필수적인 최적화인 KV 캐싱이 부족하기 때문입니다. (2) 생성 품질: MDM은 더 복잡한 언어 모델링 작업 *

최근 제안된 BD3-LM *

!

캐싱: 블록 간 KV 캐싱은 가능하지만, 블록 내 확산은 여전히 KV 지원이 부족하여 전체 속도 향상을 제한합니다.

이러한 과제를 해결하기 위해 우리는 자기회귀 및 마스크 확산 접근 방식을 융합한 새로운 언어 모델링 패러다임을 제안합니다. 우리 모델은 AR 및 MDM 목표의 조합인 하이브리드 손실로 훈련되며, 이를 통해 perplexity 면에서 두 패러다임 사이를 부드럽게 보간할 수 있습니다. 이를 위해서는 두 가지 핵심 혁신이 필요합니다. (1) AR 및 MDM 방식의 생성 모두를 지원하기 위한 디노이징 트랜스포머의 수정된 어텐션 메커니즘. (2) 확산 단계 내에서 KV 캐싱을 가능하게 하는 새로운 훈련 및 샘플링 절차 (이는 MDM에서는 이전에 사용할 수 없었던 기능입니다). 이 하이브리드 설계의 비전통적인 특성으로 인해 우리는 우리의 방법을 난해한 언어 모델(Eso-LM)이라고 명명합니다.

우리 방법은 언어 모델링 벤치마크에서 확산 모델 중 새로운 최첨단 성능을 확립하고, AR 및 MDM perplexity 사이의 보간에 대한 세밀한 제어를 가능하게 합니다. 이전 접근 방식과 달리, 확산 중 KV 캐싱을 지원하여 훨씬 빠른 추론을 제공합니다. 요약하자면, 우리의 기여는 다음과 같습니다. (1) 우리는 AR 및 MDM 패러다임을 융합하고 이전의 하이브리드 접근 방식인 BD3-LM을 능가하는 새로운 언어 모델링 프레임워크를 제안합니다. (2) 우리는 병렬 생성을 유지하면서

MDM에 KV 캐싱을 가능하게 한 최초의 접근 방식이며, 표준 MDM보다 최대 65×배, KV 캐시된 준자기회귀 기준선보다 3−4×배 빠른 추론을 달성합니다 (5.2절). (3) 우리는 BD3-LM이 낮은 샘플링 단계에서 성능이 저하됨을 보여주며, 우리 방법은 낮은 NFE 환경에서 MDM과, 높은 NFE 환경에서 AR과 경쟁력을 유지합니다 (5.3절). (4) 우리의 결과는 이산 확산에 대한 새로운 최첨단 perplexity를 확립하여 자기회귀 모델과의 격차를 좁힙니다 (5.1절).

2 배경

표기법 우리는 K 값을 가질 수 있는 스칼라 이산 확률 변수를 '원-핫(one-hot)' 열 벡터로 나타내고, $\mathcal{V}\in\{\mathbf{x}\in\{0,1\}^K:\Sigma_{i=1}^K\mathbf{x}_i=1\}$을 그러한 모든 벡터의 집합으로 정의합니다. $\mathbf{m}^K=1$을 만족하는 특별한 마스크 벡터를 $\mathbf{m}\in\mathcal{V}$라고 합시다. Δ^K가 K-심플렉스를 나타낼 때, $\pi\in\Delta^K$에 의해 주어진 확률을 가지는 K개 클래스에 대한 범주형 분포를 $\mathrm{Cat}(\cdot;\pi)$로 정의합니다. 추가적으로, $\langle\mathbf{a},\mathbf{b}\rangle$는 벡터 $\mathbf{a}$와 $\mathbf{b}$ 사이의 내적을 나타냅니다. 우리는 정렬된 집합(튜플)을 나타내기 위해 괄호 ()를 사용하고, 비정렬 집합을 나타내기 위해 중괄호 {}를 사용합니다. |A|는 집합 A의 카디널리티(기수)를 나타냅니다.

MDM은 시퀀스 순서와 디노이징 순서라는 두 가지 두드러진 순서를 특징으로 합니다. 우리는 이러한 순서들 간의 관계를 설명하기 위해 순열 σ를 사용합니다. $\mathcal{P}^L$을 $[L]=\{1,\cdots,L\}$의 모든 순열의 집합이라고 합시다. $\sigma\in\mathcal{P}^L$는 정렬된 집합(튜플)이며 전단사 함수 역할도 합니다. 즉, $\sigma(l)$는 σ의 $l$th번째 위치에 있는 항목이며, $\sigma^{-1}(i)$는 σ에서 항목 i의 위치입니다. 예를 들어, (2,4,1,3)는 {1,2,3,4}의 순서를 나타내며, $\sigma(2)=4$는 위치 2를 해당 항목 4에 매핑하고, $\sigma^{-1}(4)=2$는 항목 4를 해당 위치 2에 매핑합니다.

$\mathbf{x}\in\mathcal{V}^L$는 마스크 토큰이 없는 길이 L의 시퀀스를 나타내고, $\mathbf{x}_\ell$는 $\mathbf{x}$의 ℓth번째 항목을 나타냅니다. 우리는 '토큰 인덱스'라는 용어를 원래 순서에서 토큰의 위치를 지칭하는 데 사용합니다. 예를 들어, $\mathbf{x}_l$의 토큰 인덱스는 l입니다. $(\mathbf{z}_t)_{t\in[0,1]}\in\mathcal{V}^L$는 마스크 토큰을 포함할 수 있는 길이 L의 시퀀스를 나타냅니다. $\mathcal{M}(\mathbf{z}_t)=\{\ell|\mathbf{z}_{t\ell}=\mathbf{m}\}$는 $\mathbf{z}_t$에서 마스크 토큰의 토큰 인덱스를 나타내고, $\mathcal{C}(\mathbf{z}_t)=\{\ell|\mathbf{z}_{t\ell}\neq\mathbf{m}\}$는 $\mathbf{z}_t$에서 클린 토큰의 토큰 인덱스를 나타냅니다.

$\oplus:\mathcal{V}^m\times\mathcal{V}^n\to\mathcal{V}^{m+n}$를 길이 m 및 n인 두 시퀀스 $\mathbf{x}=(\mathbf{x}_1,\mathbf{x}_2,\cdots,\mathbf{x}_m)$과 $\mathbf{z}=(\mathbf{z}_1,\mathbf{z}_2,\cdots,\mathbf{z}_n)$에 대한 연결 연산자라고 합시다. 연결된 시퀀스 $\mathbf{x}\oplus\mathbf{z}$가 트랜스포머에 입력될 때, $\mathbf{x}$과 $\mathbf{z}$는 독립적으로 트랜스포머에 입력되었을 때와 동일한 위치 임베딩을 가집니다. $\odot:\mathcal{V}^m\times\mathcal{V}^n\to\mathcal{V}^m$를 치환 연산자라고 합시다. $m>n$을 가지는 임의의 $\mathbf{z}\in\mathcal{V}^m$와 $\mathbf{x}\in\mathcal{V}^n$에 대해, 출력 $\mathbf{y}=\mathbf{z}\odot\mathbf{x}$는 다음과 같이 주어집니다: $\mathbf{y}_{1:n}=\mathbf{x}$ 및 $\mathbf{y}_{n+1:m}=\mathbf{z}_{n+1:m}$.

2.1 자기회귀 모델

$\mathcal{V}^L$에 있는 시퀀스 $\mathbf{x}\sim q_{\text{data}}$가 주어질 때, 자기회귀(AR) 모델은 결합 분포의 다음 분해를 정의합니다: $\log p_\theta(\mathbf{x})=\Sigma_{t=1}^L\log p_\theta(\mathbf{x}_\ell|\mathbf{x}_{<\ell})$. 여기서 모델 p_θ은 일반적으로 인과적 트랜스포머 *

2.2 마스크 확산 모델

확산 모델은 $\mathcal{V}^L$의 깨끗한 데이터 $\mathbf{x}\sim q_{\text{data}}$를 $t\in[0,1]$에 대한 잠재 변수 시퀀스 $\mathbf{z}_t$으로 변환하는 전방 손상 과정 q을 역으로 학습하며, 각 변수는 $\mathbf{x}$의 점점 더 노이즈가 많은 버전을 나타냅니다 [31, 32, 8]. MDM *[26, 30, 20]*에서 전방 마스킹 과정 q은 시퀀스 $\mathbf{x}$에 걸쳐 독립적으로 분해됩니다. 즉, $q_t(.|\mathbf{x})=\Pi_\ell q_t(.|\mathbf{x}_\ell)$이며 각 토큰 $\mathbf{x}_l$는 고정된 목표 분포 $\mathrm{Cat}(.;\mathbf{m})$와 점진적으로 보간됩니다. 시간 t에서의 $\mathbf{z}_{t\ell}\sim q_t(.|\mathbf{x}_\ell)$의 주변 분포는 다음과 같습니다.

$$q_t(.|\mathbf{x}_\ell)=\mathrm{Cat}(.;\alpha_t\mathbf{x}_\ell+(1-\alpha_t)\mathbf{m}),\ (1)$$

여기서 $\alpha_t\in[0,1]$는 $\alpha_0\approx1$ 및 $\alpha_1\approx0$를 가지는 t에서 엄격하게 감소하는 함수입니다. *Sahoo et al.

[$q_{s|t}(.|₩mathbf{z}_t^{₩ell}, ₩mathbf{x}^{₩ell}) = ₩begin{cases} ₩text{Cat}(.; ₩mathbf{z}_t^{₩ell}) & ₩mathbf{z}_t^{₩ell} ₩neq ₩mathbf{m}, \\ ₩text{Cat}₩left(:{₩frac{(1-₩alpha_s)₩mathbf{m} + ({₩alpha_s - ₩alpha_t}) ₩mathbf{x}^{₩ell}}{1-₩alpha_t}}₩right) & ₩mathbf{z}_t^{₩ell} = ₩mathbf{m}. ₩end{cases}$] (2)

디노이징 모델 $\mathbf{x}\theta: \mathcal{V}^L \to (\Delta K)^L$가 주어질 때, 시퀀스 $\mathbf{z}_s$에 걸친 역 마스크 해제 과정 $p\theta(.|\mathbf{z}_t)_{s|t}$은 다음과 같이 매개변수화 됩니다.

$$p\theta(.|\mathbf{z}_t)_{s|t} = \Pi_{\ell} L p\theta^{\ell}(.|\mathbf{z}_t)_{s|t} = \Pi_t L q_{s|t}^{\ell}(.|\mathbf{z}_t^{\ell}, \mathbf{x}^{\ell} = \mathbf{x}\theta^{\ell}(\mathbf{z}_t)). \quad (3)$$

*Sahoo et al.

$$\mathscr{L}\text{MDM}(\mathbf{x}) = \mathbb{E}_{q,t\sim[0,1]} \alpha_t' 1 - \alpha_t \Sigma_{\ell \in \mathscr{M}(\mathbf{z}_t)} \log \langle \mathbf{x}\theta^{\ell}(\mathbf{z}_t), \mathbf{x}^{\ell} \rangle, \quad (4)$$

이는 마스크된 토큰 $\mathscr{M}(\mathbf{z}_t)$에 대해서만 계산되는 마스크 언어 모델링 손실 *

길이가 L인 시퀀스를 생성하기 위해, 역확산 과정은 $\mathbf{z}_{t=1}$에서 시작하며, 여기서 $\ell = 1, \cdots, L$에 대해 $\mathbf{z}_{t=1}^{\ell} = \mathbf{m}$이다. 이 과정은 T 단계로 진행되며, 각 $\mathbf{z}_{s^{\ell}}$는 (3)에서 정의된 $p\theta(.|\mathbf{z}_t)_{s|t}$에서 독립적으로 샘플링된다. 한 위치가 마스크 해제되면, 그 위치는 고정된 상태로 유지된다. 여러 토큰이 병렬로 디노이즈될 수 있으므로, 총 네트워크 함수 평가(NFE) 수는 L 미만이 될 수 있어 더 빠른 생성을 가능하게 한다. 그러나 $\mathbf{x}\theta(\mathbf{z}_t)$의 양방향 트랜스포머로 인해 각 NFE는 비용이 많이 든다.

2.3 블록 디노이징 확산 이산 언어 모델

블록 디노이징 확산 이산 언어 모델(BD3-LM) *

블록의 크기를 변경함으로써, BD3-LM은 AR 모델과 MDM 사이를 보간한다. BD3-LM은 $\mathbf{x}$의 토큰들을 $B = L/L'$과 함께 L'개의 연속적인 토큰으로 구성된 B개의 블록으로 그룹화하며, 여기서 B는 정수라고 가정한다. $\mathbf{x}$에 대한 우도는 블록에 대해 $-\log p\theta(\mathbf{x}) = -\Sigma_{b=1}^{B} \log p\theta(\mathbf{x}^b|\mathbf{x}^{<b}) \leq \Sigma_{b=1}^{B} \mathscr{L}\text{MDM}(\mathbf{x}^b; \mathbf{x}^{<b})$으로 자동회귀적으로 인수분해되며, 여기서 $p\theta(\mathbf{x}^b|\mathbf{x}^{<b})$은 조건부 MDM이고 $\mathscr{L}\text{MDM}(\mathbf{x}^b; \mathbf{x}^{<b})$은 (4)에서 정의된 MDLM에 대한 NELBO로, 블록에 걸쳐 순차적으로 적용된다.

3 난해한 언어 모델

우리는 언어 모델링을 위한 새로운 패러다임인 난해한 언어 모델(Eso-LM)을 제안하는데, 이는 AR 모델과 MDM의 상호 보완적인 조합을 이룬다. 우리의 접근 방식은 두 가지 주요 구성 요소를 가진다: (1) 순차적(AR 방식) 및 병렬(MDM 방식) 생성을 모두 지원하는 모델군을 소개하고; (2) 섹션 4에서는 이러한 하이브리드 생성을 가능하게 하는 트랜스포머 *

AR 모델은 현재 언어 모델링 성능에서 선두를 달리고 있지만, 토큰을 순차적으로 생성하여 추론 속도가 느리다. 대조적으로, MDM은 여러 토큰을 병렬로 생성하며 더 제어 가능하지만 [29, 19], 일반적으로 AR 모델보다 더 높은 혼란도(perplexity)를 보인다 [26, 27]. 핵심 질문은: 그들의 강점을 결합할 수 있는가? 이다. 우리는 일부 토큰은 MDM을 통해 병렬로 생성되고 나머지는 좌에서 우로 순차적으로 생성되는 하이브리드 접근 방식을 도입한다. 이로 인해 두 가지 주요 질문이 발생한다. 첫째, 이러한 생성 모델의 우도를 계산할 수 있는가? 우리는 Eso-LM이 진정한 우도에 대한 원리적인 경계를 허용함을 보여준다. 둘째, 트랜스포머의 어텐션 메커니즘을 두 가지 생성 스타일을 모두 지원하도록 어떻게 조정할 수 있는가? 훈련 중, AR 모델에서는 깨끗한(clean) 시퀀스에 인과적 어텐션이 적용되는 반면, MDM은 마스크된 토큰과 마스크 해제된 토큰 모두에 대한 양방향 어텐션을 사용한다. 섹션 4에서는 트랜스포머를 위한 맞춤형 어텐션 메커니즘을 설계하여 이 두 가지 상이한 아키텍처를 조화시킨다. 또한, 우리는 어텐션 행렬에 희소성을 도입하여 MDM을 위한 KV-캐싱을 가능하게 하며, 이는 긴 컨텍스트 생성 속도를 크게 향상시킨다.

3.1 자동회귀 모델과 마스크된 확산 융합

데이터 분포에서 추출한 샘플을 $\mathcal{V}^L$ 내의 $\mathbf{x}\sim q_{data}(\mathbf{x})$라고 하고, θ으로 매개변수화된 우리 모델 분포를 p_θ라고 하자. Eso-LM은 p_θ를 두 가지 구성 요소로 분해한다: AR 모델 p_θ^{AR}와 MDM p_θ^{MDM}. MDM은 부분적으로 마스크된 시퀀스 $\mathbf{z}_0\in\mathcal{V}^L\sim p_\theta^{MDM}(\mathbf{z}_0)$을 생성하고, AR 모델은 나머지 마스크 해제 단계를 자동회귀적인 좌에서 우로 진행한다: $p_\theta^{AR}(\mathbf{x}|\mathbf{z}_0)$. 이러한 하이브리드 생성 과정의 한계 우도(marginal likelihood)는 다음과 같다:

$$p_\theta(\mathbf{x})=\Sigma_{\mathbf{z}_0\in\mathcal{V}^L}p_\theta^{AR}(\mathbf{x}|\mathbf{z}_0)p_\theta^{MDM}(\mathbf{z}_0). \quad (5)$$

이 합계는 다루기 어렵지만, 사후 분포 $q(\mathbf{z}_0|\mathbf{x})$ *

$$-\log p_\theta(\mathbf{x}) \le -\mathbb{E}_{\mathbf{z}_0\sim q_0(\cdot|\mathbf{x})}[\log p_\theta^{AR}(\mathbf{x}|\mathbf{z}_0)]+D_{KL}(q_0(\mathbf{z}_0|\mathbf{x}) \| p_\theta^{MDM}(\mathbf{z}_0)) \quad (6)$$
$$=-\mathbb{E}_{\mathbf{z}_0\sim q_0(\cdot|\mathbf{x})}[\Sigma_{\ell\in\mathcal{M}(\mathbf{z}_0)}\log p_\theta^{AR}(\mathbf{x}^\ell|\mathbf{z}_0,\mathbf{x}^{<\ell})]+D_{KL}(q_0(\mathbf{z}_0|\mathbf{x}) \| p_\theta^{MDM}(\mathbf{z}_0)).$$

여기서 $p_\theta^{AR}(\mathbf{x}|\mathbf{z}_0)$은 마스크된 위치 $\ell\in\mathcal{M}(\mathbf{z}_0)$에 대한 결합 AR 우도이며, $\mathbf{z}_0$에 있는 깨끗한 토큰에 조건화된다. AR의 경우, 디노이징 네트워크 $\mathbf{x}_\theta:\mathcal{V}^L\to(\Delta^K)^L$은 입력 $\mathbf{z}_0\odot\mathbf{x}^{<\ell}$에 대해 작동하며, 여기서 치환 연산자 $\odot$는 $\mathbf{z}_0$의 첫 $l-1$개 토큰을 $\mathbf{x}^{<\ell}$로 대체한다. 각 $\ell\in\mathcal{M}(\mathbf{z}_0)$에 대해, $\mathbf{x}_\theta^\ell(\mathbf{z}_0\odot\mathbf{x}^{<\ell})$은 $\mathbf{x}^{<\ell}$와 $\mathbf{z}_0$이 주어졌을 때 깨끗한 토큰 $\mathbf{x}^\ell$의 분포를 근사하며, $\mathbf{z}_0$은 위치 ℓ를 넘어선 깨끗한 토큰을 포함할 수 있다. 부록 B.1에서는 KL 항을 분석하고 NELBO가 다음과 같음을 보여준다:

$$\mathcal{L}_{NELBO}(\mathbf{x})=\mathbb{E}_{\mathbf{z}_0\sim q_0}[-\Sigma_{\ell\in\mathcal{M}(\mathbf{z}_0)}\log\langle\mathbf{x}_\theta^\ell(\mathbf{z}_0\odot\mathbf{x}^{<\ell}),\mathbf{x}^\ell\rangle]}_{\text{AR loss}}+\int_{t=0}^{t=1}\alpha_t' 1-\alpha_t\mathbb{E}_{\mathbf{z}_t\sim q_t}[\Sigma_{\ell\in\mathcal{M}(\mathbf{z}_t)}\log\langle\mathbf{x}_\theta^\ell(\mathbf{z}_t),\mathbf{x}^\ell\rangle]}_{\text{M}} \quad (7)$$

AR과 확산 사이 보간

$\alpha_0=1$일 때, 사후 샘플은 $\mathbf{z}_0=\mathbf{x}$이고 모든 토큰은 MDM에 의해 생성된다; 따라서 (7)에서 AR 손실은 0이 되고, $\mathcal{L}_{NELBO}$는 MDM 손실로 축소된다. 반대로, $\alpha_0=0$일 때, 모든 토큰은 마스크되고 MDM 손실은 사라져 $\mathcal{L}_{NELBO}$는 AR 손실로 축소된다. 따라서 Eso-LM은 하이퍼파라미터 α_0에 의해 제어되는 AR 및 MDM 패러다임 사이를 보간한다.

3.2 샘플링

우리는 (5)에 정의된 2단계 샘플링을 수행한다: $p_\theta(\mathbf{x})$에서 샘플링하기 위해, 먼저 부분적으로 마스크된 시퀀스 $\mathbf{z}_0\sim p_\theta^{MDM}$을 그린 다음 p_θ^{AR}를 사용하여 완성한다.

확산 단계

완전히 마스크된 시퀀스 $\mathbf{z}_{t=1}=\mathbf{m}^{1:L}$에서 시작하여, 표준 선조 샘플링(섹션 2.2)은 각 단계에서 마스크된 토큰의 부분 집합만 업데이트하면서 전체 시퀀스에 대한 정방향 패스를 수행하여 FLOP를 낭비한다. 효율성을 향상시키기 위해, 우리는 표준 샘플링 및 훈련 절차에 두 가지 주요 변경 사항을 제안한다. 샘플링 동안, 우리는 확산 디노이징 스케줄 $\mathcal{S}^{MDM}=(S^1,\cdots,S^{1/T})$을 미리 계산하는데, 여기서 S^t는 확산 단계 t에서 디노이즈될 마스크 토큰 인덱스의 집합이다. 전체 시퀀스를 처리하는 대신, 우리는 부분 시퀀스 $\{\mathbf{z}_t^\ell|\ell\in\mathcal{C}(\mathbf{z}_t)\cup S^t\}$—즉, 깨끗한 토큰과 디노이징이 예정된 토큰—에 대해서만 정방향 패스를 수행하여, 특히 긴 시퀀스에서 계산을 상당히 줄인다. 이 샘플링 방식과 일치시키기 위해, 모델은 훈련 시 마스크 토큰에 대한 양방향 어텐션을 피해야 한다 (섹션 4에서 논의된 바와 같이). 이 접근 방식은 또한 확산 동안 효율적인 KV-캐싱을 가능하게 한다 (섹션 4.2에서 논의된 바와 같이).

순차 단계

우리는 최종 출력 $\mathbf{x}\sim$을 $\mathbf{z}_0$로 초기화한다. 그 다음 AR 모델은 순차적 디노이징 스케줄 $\mathcal{S}^{AR}=((i)|i\in\mathcal{M}(\mathbf{z}_0))$를 사용하여 마스크 토큰을 좌에서 우로 채우는데, 여기서 우리는 단일 요소 집합이 그들의 유일한 요소에 의해 오름차순으로 정렬되

도록 요구한다. 표준 AR 디코딩과 달리, 각 $\mathbf{x}\sim\ell$은 완전히 깨끗한 토큰으로 구성된 왼쪽 컨텍스트와 $\mathbf{z}0$의 오른쪽 깨끗한 토큰 모두에 조건화되어 더 풍부한 생성을 가능하게 한다. 우리는 오른쪽 마스크 토큰에 대한 평가를 건너뛰어 불필요한 계산을 줄인다. 순차 단계는 자연스럽게 KV 캐싱을 지원한다.

우리는 통합 디노이징 스케줄을 $\mathcal{S}=\mathcal{S}$MDM$\cup\mathcal{S}$AR으로 나타내며, 이는 두 샘플링 스케줄을 연결하여 [L]을 분할한다. $\alpha0=1$일 때, 모든 토큰은 확산에 의해 생성되므로, $\mathcal{S}=\mathcal{S}$MDM이고 $\mathcal{S}$AR$=\varnothing$이다 (공집합). $\alpha0=0$일 때, 모든 토큰은 순차적으로 생성되므로, $\mathcal{S}=\mathcal{S}$AR이고 $\mathcal{S}$MDM$=\varnothing$이다. 전체 샘플링 알고리즘 및 예시는 부록 B.4를 참조하라.

4 공유 디노이징 트랜스포머를 위한 어텐션 메커니즘 설계

이 섹션에서는 공유 트랜스포머 아키텍처를 사용하여 순차적(AR) 및 병렬(MDM) 생성 모드를 모두 지원하는 통합 모델링 프레임워크를 소개한다. 우리의 핵심 기술적 기여는 인과적 어텐션과 한 칸 이동 디코딩을 필요로 하는 AR 모델과 양방향 어텐션에 의존하는 MDM 사이의 아키텍처 불일치를 조화시키는 유연한 어텐션 메커니즘이다. 이를 달성하기 위해, 우리는 L'이 시퀀스 길이를 나타내는 어텐션 편향(bias) 행렬 $A\in\{-\infty,0\}^{L'\times L'}$을 도입하여 표준 어텐션을 다음과 같이 조절한다:

₩textscSelf−Attention$(Q,K,V,A)=$softmax$(QK'd+A)V$

여기서 $Q,K,V\in\mathbb{R}^{L'\times d}$는 쿼리, 키, 값 행렬을 나타낸다. A의 항목들은 정보 흐름을 제어한다: $Ai,j=0$는 토큰 i에서 j로의 어텐션을 "허용"하고 $Ai,j=-\infty$은 "차단"한다. 이 메커니즘은 단일 트랜스포머가 필요에 따라 인과적 및 양방향 동작을 모두 에뮬레이트할 수 있도록 한다.

이 통합 어텐션 스키마를 기반으로, 우리는 두 가지 변형인 Eso-LM (A)와 Eso-LM (B)를 제안한다. Eso-LM (A)는 어텐션을 희소화하고 각 확산 단계에서 마스크 토큰의 부분 집합에 디노이징 트랜스포머를 선택적으로 적용함으로써 계산량 감소에 중점을 둔다. Eso-LM (B)는 마스크 토큰뿐만 아니라 깨끗한 토큰에도 인과적 마스킹을 도입하여 이 아이디어를 더욱 확장하고, 이는 KV-캐싱을 가능하게 하여 더 큰 속도 향상을 가져온다—다만 혼란도(perplexity)가 약간 감소하는 대가가 따른다. 이 섹션의 나머지 부분에서는 이러한 통합을 가능하게 하는 어텐션 공식을 설명하고 확산 단계와 순차 단계 모두에서 이 두 변형 간의 차이점을 상세히 다룬다.

확산 단계

표준 MDM은 샘플링 중에 두 가지 비효율성을 겪는다: (i) 부분 업데이트에 대해서도 전체 시퀀스 정방향 패스를 수행하며, (ii) 양방향

그림 2: 확산 단계 학습을 위한 어텐션 바이어스 비교. 주황색은 0(어텐션)을 나타내고 회색은 $-\infty$(어텐션 없음)을 나타냅니다. 클린 시퀀스는 $\mathbf{x}$=(A,B,C,D,E,F)입니다. 무작위 마스킹 후, $\mathbf{z}t$=(A,M,C,M,M,F)를 얻습니다. 정수는 $\mathcal{M}(\mathbf{z}t)=\{2,4,5\}$ 및 $\mathcal{C}(\mathbf{z}t)=\{1,3,6\}$와 함께 위치 인덱스를 나타냅니다. 무작위 순서는 마스크 토큰 이전에 클린 토큰이 오는 $\sigma=(3,1,6,4,5,2)\sim\mathcal{P}6$입니다. 빨간색은 Eso-LM (A)와 Eso-LM (B) 간의 차이를 강조합니다.

어텐션. 섹션 3.2에서 간략히 논의했듯이, 우리는 (i)을 단계 t에서 $\{\mathbf{z}ti|i\in\mathcal{C}(\mathbf{z}t)\cup St\}$에 대해서만 $\mathbf{x}$R을 평가함으로써 해결합니다. 이 가능성을 열기 위해, 우리는 AO-ARM

순차 단계 순차 생성 중에 모델은 $\mathbf{z}0\sim p\theta$MDM(.)에 의해 생성된 마스크 토큰 $(\mathbf{z}0i)t\in\mathcal{M}(\mathbf{z}0)$을 왼쪽에서 오른쪽으로 노이즈 제거해야 합니다. 이는 생성 중에는 간단하지만 학습 중에는 어렵습니다. 각 토큰이 왼쪽만 어텐션하는 표준 AR 모델과 달리, 각 마스크 토큰 $\mathbf{z}0i$는 (i) 자기 자신(위치 임베딩을 위해), (ii) 클린 왼쪽 컨텍스트 $\mathbf{x}$<i($\mathbf{z}0$<i의 모든 이전 마스크 토큰이 채워졌다고 가정), (iii) 그리고 오른쪽 컨텍스트 $\{\mathbf{x}j|j\in\mathcal{C}(\mathbf{z}0),j$>i} 또는 동등하게 $\{\mathbf{z}0j|j\in\mathcal{C}(\mathbf{z}0),j$>i} 의 클린 토큰에 어텐션해야 합니다. 학습 중 (ii)를 처리하는 것은 간단하지 않습니다. 왜냐하면 $\mathbf{z}0$의 각 마스크 토큰은 자신의 왼쪽에 있는 토큰이 $p\theta$AR에 의해 이미 노이즈 제거되었다고 가정하기 때문입니다. 이 동작을 시뮬레이션하기 위해, 우리는 학습 중에 연결된 시퀀스 $\mathbf{z}0\oplus\mathbf{x}$를 트랜스포머에 입력하고 모든 마스크 토큰에 대해 올바른 정보 흐름을 강

제하는 사용자 정의 어텐션 마스크 A를 구성합니다. 손실은 (7)에 따라 $\mathbf{z}0$의 마스킹된 토큰에 해당하는 로짓에 대해서만 계산됩니다. 생성 중에는 이 문제가 자연스럽게 해결되며, 연결이 필요하지 않습니다.

4.1 Eso-LM (A)

우리는 확산 단계에서 노이즈 제거 트랜스포머 내 마스크 토큰에 대한 양방향 어텐션을 제거하는 Eso-LM (A)를 소개합니다. 이 수정은 섹션 3.2에 설명된 샘플링 스케줄 사용을 가능하게 하여 더 빠른 추론을 허용합니다. Eso-LM (A)는 순차 샘플링 중에만 KV 캐싱을 지원하지만, 확산 샘플링 중에는 표준 MDM보다 훨씬 적은 계산을 수행합니다.

4.1.1 학습

확산 단계 노이즈 제거 트랜스포머는 노이즈 제거할 마스크 토큰을 포함하는 $\mathbf{z}t\sim q_t(.|\mathbf{x})$을 입력으로 받고, $\mathbf{x}$을 목표로 받습니다. σ에서 $\mathbf{z}t$의 클린 토큰이 $\mathbf{z}t$의 마스크 토큰보다 선행한다는 자연스러운 제약과 함께 무작위 순서 $\sigma\sim\mathcal{P}L$가 샘플링됩니다. 우리는 L×L 어텐션 바이어스를 다음과 같이 정의합니다.

$$A_{i,j}=\begin{cases}0 & \forall (i,j)\in C(z_t)\times C(z_t) \text{ i f } \sigma^{-1}(i)\geq\sigma^{-1}(j)\\ -\infty & \forall (i,j)\in M(z_t)\times[L] \text{ o t h e r w i s e .}\end{cases}\tag{10}$$

클린 토큰 $\{\mathbf{z}t_i|i\in\mathcal{C}(\mathbf{z}t)\}$은 서로 양방향 어텐션을 가지며 (8), 마스크 토큰 $(\mathbf{z}t_i)i\in\mathcal{M}(\mathbf{z}t)$은 σ에 따라 클린 토큰, 자기 자신, 그리고 이전 마스크 토큰에 어텐션합니다 (9). 우리는 다음을 무시한다는 점에 유의하십시오.

σ의 클린 토큰들 간의 순서를 무시하고 단순히 그들 간의 양방향 어텐션을 사용합니다. 예시는 그림 2를 참조하십시오. 흥미롭게도, A의 행과 열을 σ (그림 7)에 따라 정렬하면 A은 Prefix-LM

순차 단계 노이즈 제거 트랜스포머는 연결된 시퀀스 $\mathbf{z}0\oplus\mathbf{x}\in\mathcal{V}2L$을 입력으로 받는데, 여기서 $\mathbf{z}0\sim q_0(.|\mathbf{x})$은 노이즈 제거할 마스크 토큰을 포함하고, $\mathbf{x}$는 목표입니다. 우리는 2L×2L 어텐션 바이어스를 다음과 같이 정의합니다.

$A_{i,j}=0$	$M(z_0)\times M(z_0)$의 모든 (i,j)에 대해 i=j인 경우	(11)
$A_{i,j+L}=0$	$M(z_0)\times C(z_0)$의 모든 (i,j)에 대해	(12)
$A_{i,j+L}=0$	$M(z_0)\times M(z_0)$의 모든 (i,j)에 대해 i>j인 경우	(13)
$A_{i+L,j+L}=0$	$C(z_0)\times C(z_0)$의 모든 (i,j)에 대해	(14)
$A_{i+L,j+L}=0$	$M(z_0)\times C(z_0)$의 모든 (i,j)에 대해	(15)
$A_{i+L,j+L}=0$	$M(z_0)\times M(z_0)$의 모든 (i,j)에 대해 i≥j인 경우	(16)
$A_{i,j}=-\infty$	그 외의 경우.	(17)

노이즈가 있는 컨텍스트 $\mathbf{z}0$의 토큰에 대한 어텐션 메커니즘은 다음과 같이 설명됩니다. 마스크 토큰 $(\mathbf{z}0_i)i\in\mathcal{M}(\mathbf{z}0)$은 (i) 자기 자신 (11), (ii) 클린 토큰 $\{\mathbf{x}_j|j\in\mathcal{C}(\mathbf{z}0)\}$ (12), 그리고 (iii) 자신의 왼쪽에 있는 마스크 토큰의 클린 버전 $\{\mathbf{x}_j|j\in\mathcal{M}(\mathbf{z}0),i>j\}$ (13)에 어텐션합니다. 클린 토큰 $(\mathbf{z}0_i)i\in\mathcal{C}(\mathbf{z}0)$는 다른 어떤 토큰도 어텐션하지 않으므로 (17) 무엇이든 어텐션할 수 있습니다. 클린 컨텍스트 $\mathbf{x}0$의 토큰에 대한 어텐션 메커니즘은 다음과 같이 설명됩니다. 토큰 $\{\mathbf{x}_i|i\in\mathcal{C}(\mathbf{z}0)\}$은 양방향 어텐션을 가집니다 (14). 마스크 토큰에 해당하는 클린 토큰 $(\mathbf{x}_i)i\in\mathcal{M}(\mathbf{z}0)$은 $\{\mathbf{x}_j|j\in\mathcal{C}(\mathbf{z}0)\}$ (15) 및 $\{\mathbf{x}_j|j\in\mathcal{M}(\mathbf{z}0),i\geq j\}$ (16)에 어텐션합니다.

예시는 그림 6을 참조하십시오. σ가 두 가지 속성을 가진 순서라고 합시다: (i) σ에서 $\mathbf{z}0$의 클린 토큰이 $\mathbf{z}0$의 마스크 토큰보다 선행하고, (ii) σ에서 $\mathbf{z}0$의 마스크 토큰이 자연 순서로 배열됩니다. 다시 말하지만, 클린 토큰 $\{\mathbf{x}_i|i\in\mathcal{C}(\mathbf{z}0)\}$ 간의 순서는 무시되며 우리는 그들 간의 양방향 어텐션을 사용합니다. 4개의 LxL 블록 각각의 행과 열이 σ에 따라 정렬될 때, A는 구현에서 활용하는 희소하고 패턴화된 블록 구조(그림 8)를 나타냅니다 (보충 자료 B.5).

4.1.2 샘플링

확산 또는 순차 샘플링 중에, 부분적으로 마스킹된 시퀀스 $\mathbf{z}_k$가 주어졌을 때, 노이즈 제거 모델은 $K=|\mathcal{S}|$인 $S_k \in S=\{S^1,\cdots,S^K\}$에 대해 마스크 토큰 $\{\mathbf{z}_{ki}|i\in S_k\}$의 노이즈를 제거해야 합니다. 우리는 토큰들의 부분 집합 $\{\mathbf{z}_{ki}|i\in\mathcal{C}(\mathbf{z}_k)\cup S_k\}$에 대해 순방향 패스를 수행합니다. 토큰들의 부분 집합에 대해 순방향 패스를 수행하는 동안, 실제 시퀀스에서 이 토큰들의 위치 임베딩이 보존된다는 점에 유의하는 것이 중요합니다. 아래에서는 순방향 패스에 사용되는 어텐션 바이어스를 논의합니다.

DkMDM를 단계 k 이전에 확산 단계에서 디코딩된 토큰들의 인덱스 집합이라고 하고, DkAR를 순차 단계에 대한 것이라고 합시다. 정의된 S에 따라 토큰을 노이즈 제거하는 순서를 σ이라고 합시다. 우리는 단계 k에서의 L×L 어텐션 바이어스를 다음과 같이 정의합니다.

$A_{i,j}=$	**0**	**DkMDM×DkMDM의 모든 (i,j)에 대해**	**(18)**
	0	DkAR×DkMDM의 모든 (i,j)에 대해	(19)
	0	DkAR×DkAR의 모든 (i,j)에 대해 $i\geq j$인 경우	(20)
	0	Sk×(DkMDM∪DkAR)의 모든 (i,j)에 대해	(21)
	0	Sk×Sk의 모든 (i,j)에 대해 $\sigma^{-1}(i)\geq\sigma^{-1}(j)$인 경우	(22)
	-∞	그 외의 경우.	(23)

확산 중에 디코딩된 클린 토큰 $\{\mathbf{z}_{ki}|i\in\text{DkMDM}\}$은 서로 양방향 어텐션을 가집니다 (18). 순차적으로 디코딩된 클린 토큰 $(\mathbf{z}_{ki})_{i\in\text{DkAR}}$은 확산 중에 디코딩된 클린 토큰 $\{\mathbf{z}_{kj}|j\in\text{DkMDM}\}$ (19), 자기 자신, 그리고 이전의 순차적으로 디코딩된 클린 토큰 $\{\mathbf{z}_{kj}|j\in\text{DkAR},i>j\}$ (20)에 어텐션합니다. 노이즈 제거할 마스크 토큰 $(\mathbf{z}_{ki})_{i\in S_k}$는 모든 디코딩된 클린 토큰 $\{\mathbf{z}_{kj}|j\in\text{DkMDM}\cup\text{DkAR}\}$ (21), 자기 자신, 그리고 σ에 따라 이전의 노이즈 제거할 마스크 토큰 $\{\mathbf{z}_{kj}|j\in S_k,\sigma^{-1}(i)>\sigma^{-1}(j)\}$ (22)에 어텐션합니다. 노이즈 제거 예정이 아닌 마스크 토큰 $(\mathbf{z}_{ki})_{i\in S>k}$은 다른 어떤 토큰도 어텐션하지 않으므로 (23) 무엇이든 어텐션할 수 있습니다.

(i) $(\mathbf{z}_{ki})_{i\in\mathcal{C}(\mathbf{z}_k)\cup S_k}$가 $\{\mathbf{z}_{ki}|i\in S>k\}$에 어텐션하지 않고 (ii) 클린 토큰을 샘플링하기 위해 $\{\mathbf{z}_{ki}|i\in S_k\}$에 대한 트랜스포머 출력만 사용하므로, $\{\mathbf{z}_{ki}|i\in S>k\}$에 대한 어떤 계산도 피할 수 있어 확산 단계에서 계산량을 크게 줄일 수 있습니다. 그림 9는 이 샘플링 절차가 순차 단계 동안 KV 캐싱을 사용하여 예시 시퀀스를 생성하는 방법을 보여줍니다.

8

4.2 Eso-LM (B)

Eso-LM (A)는 순차 샘플링 중에는 KV 캐싱을 수행할 수 있지만, 클린 토큰들이 서로 양방향 어텐션을 가지므로 확산 샘플링 중에는 그렇게 할 수 없습니다. 따라서 마스크 토큰 세트가 노이즈 제거된 후, 이제 새로 노이즈 제거된 토큰을 포함한 모든 클린 토큰에 대한 KV가 업데이트되어야 합니다. 확산 샘플링 중에 KV 캐싱을 가능하게 하기 위해, 우리는 클린 토큰들 사이에 인과적 어텐션을 강제하며, 이는 Eso-LM (B)로 이어집니다. 우리는 또한 두 단계 사이에 공유 KV 캐시를 가능하게 합니다.

4.2.1 학습

확산 단계 노이즈 제거 트랜스포머는 노이즈 제거할 마스크 토큰을 포함하는 $\mathbf{z}_t\sim q_t(.|\mathbf{x})$을 입력으로 받고, $\mathbf{x}$을 목표로 받습니다. 우리는 MDM과 AO-ARM

$$A_{i,j}=\{0\amp;\text{if }\sigma^{-1}(i)\geq\sigma^{-1}(j)\;\forall\,(i,j)\in[L]\times[L]-\infty\amp;\text{otherwise.}\tag{24}$$

예시는 그림 2를 참조하십시오. 또한, A의 행과 열을 σ (그림 7)에 따라 정렬하면 A은 인과적 어텐션 바이어스가 됩니다; 우리는 이 속성을 구현에 활용합니다 (보충 자료 B.5).

순차 단계 노이즈 제거 트랜스포머는 연결된 시퀀스 $\mathbf{z}0 \oplus \mathbf{x} \in \mathcal{V}2L$을 입력으로 받습니다. 여기서 $\mathbf{z}0 \sim q0(.|\mathbf{x})$은 노이즈 제거할 마스크 토큰을 포함하고, $\mathbf{x}$는 타겟입니다. 샘플링 동안, 인과적 어텐션이 적용되고 KV 캐시가 구축되는 $\mathbf{z}0 \sim p\phi$MDM의 깨끗한 컨텍스트는 확산 단계에 의해 제공됩니다. 확산 단계에서 구축된 KV 캐시를 효과적으로 사용하기 위해, 순차 단계 훈련 동안 깨끗한 토큰 $\{\mathbf{x}i|i \in \mathcal{C}(\mathbf{z}0)\}$ 사이에 유사한 인과적 어텐션을 강제해야 합니다. 이를 달성하기 위해, 우리는 (i) σ에서 $\mathbf{z}0$의 깨끗한 토큰이 $\mathbf{z}0$의 마스크 토큰보다 선행하고 (ii) σ에서 마스크 토큰이 자연스러운 순서로 있다는 제약 조건으로 $\sigma \sim \mathcal{P}L$를 먼저 샘플링합니다. 그런 다음, 우리는 (11)-(17)을 재사용하지만, $\{\mathbf{x}i|i \in \mathcal{C}(\mathbf{z}0)\}$ (14)에 대한 양방향 어텐션을 σ (29)에 따른 인과적 어텐션으로 대체합니다.

Ai,j=0if i=j∀(i,j)∈$\mathcal{M}$(z0)×$\mathcal{M}$(z0)₩tag26

Ai,j+L=0∀(i,j)∈$\mathcal{M}$(z0)×$\mathcal{C}$(z0)₩tag27

Ai,j+L=0if i>j∀(i,j)∈$\mathcal{M}$(z0)×$\mathcal{M}$(z0)₩tag28

Ai+L,j+L=0if σ−1(i)≥σ−1(j)∀(i,j)∈$\mathcal{C}$(z0)×$\mathcal{C}$(z0)₩tag29

Ai+L,j+L=0∀(i,j)∈$\mathcal{M}$(z0)×$\mathcal{C}$(z0)₩tag30

Ai+L,j+L=0if i≥j∀(i,j)∈$\mathcal{M}$(z0)×$\mathcal{M}$(z0)₩tag31

Ai,j=−∞otherwise.₩tag32

예시는 그림 6을 참조하십시오. 네 개의 L×L 블록 각각의 행과 열이 σ에 따라 정렬되면, A는 우리가 구현에서 활용하는 구조(그림 8)를 보여줍니다 (부록 B.5).

4.2.2 샘플링

4.1.2절과 동일한 설정 및 표기법에 따라, 우리는 (18)-(23)을 재사용하지만, DkMDM (18)에 대한 양방향 어텐션을 σ에 따른 인과적 어텐션, 즉 노이즈 제거 순서 (33)로 대체합니다.

Ai,j={0amp;if σ−1(i)≥σ−1(j)∀(i,j)∈DkMDM×DkMDM0amp;∀(i,j)∈DkAR×DkMDM0amp;if i≥j∀(i,j)∈DkAR×DkAR0amp
₩tag38

두 단계에서의 KV 캐싱 여기서, 4.1.2절에서와 같이, 우리는 미래 마스크 토큰 $\{\mathbf{z}ki|i \in S\>k\}$에 대한 계산을 수행하지 않습니다. 또한, 이전에 디코딩된 토큰들은 확산 방식이든 순차 방식이든 상관없이 그들 사이에 인과적 어텐션을 가지고 있기 때문에, k 단계에서 새로 생성된 토큰들은 k+1 단계에서 KV 캐시를 구축하는 데 사용될 수 있으며, 캐시된 KV 값은 k+2 단계부터 사용될 수 있습니다. 그림 1은 이 샘플링 절차가 어떻게 예시 시퀀스를 생성하는지 보여줍니다.

---

5. 실험

우리는 두 가지 표준 언어 모델링 벤치마크, 즉 One Billion Words 데이터셋(LM1B) *

5.1 우도 평가

우리의 실험은 Eso-LM이 LM1B 및 OWT에서 확산 모델 중 최첨단 퍼플렉시티를 달성하며, MDM과 AR 퍼플렉시티 간의 더 미세한 보간을 제공함을 보여줍니다 (표 1 및 표 2).

실험 설정 우리는 주요 마스크 확산 모델인 MDLM *

결과 우리 방법의 경우, PPL은 (7)을 사용하여 계산됩니다. LM1B에서, 우리는 Eso-LM (A)를 $\alpha 0 \in \{0.0625, 0.125, 0.25, 0.5, 1.0\}$으로, Eso-LM (B)를 $\alpha 0 = 1$으로 훈련합니다. 우리는 Eso-LM (A)가 MDLM과 AR 퍼플렉시티 사이를 효과적으로 보간한다는 것을 발견했습니다. 놀랍게도, 마스크 토큰에 대한 양방향 어텐션을 인과적 마스크로 대체하여 MDM을 수정한 Eso-LM (A)는 $\alpha 0 = 1$에서 MDLM보다 ≈ 1 PPL만큼 뛰어난 성능을 보입니다. 이러한 개선은 앞서 설명한 낮은 분산 훈련 손실에 기인할 수 있습니다. 대조적으로, Eso-LM (B)는 깨끗한 토큰들 사이에 인과적 의존성을 도입하여, $\alpha 0 = 1$에서 Eso-LM (A)에 비해 성능을 ≈ 4 PPL만큼 저하시킵니다. 이는 Eso-LM (A)의 깨끗한 토큰에 대한 양방향 어텐션이 Eso-LM (B)의 인과적 어텐션보다 엄격하게 더 표현력이 좋기 때문에 예상된 결과입니다. OWT에서는, Eso-LM (B)가 확산 단계 동안 KV 캐싱을 지원하는 유일한 모델(BD3-LM 포함)이므로 특히 설득력이 있습니다. $\alpha 0 \in \{0.0625, 0.125, 0.25, 0.5, 1.0\}$로 훈련되었을 때, AR과 MDM 퍼플렉시티 사이의 부드러운 보간을 보여줍니다 (표 2). LM1B와 마찬가지로, Eso-LM (A)는 OWT에서 MDLM과 경쟁력을 유지합니다.

5.2 생성 속도

우리의 정성적 평가 실험은 Eso-LM이 KV 캐싱을 지원하는 이전 확산 기반 방법보다 3–4×배 빠르며, KV 캐싱을 지원하지 않는 MDM보다 최대 65×배 빠르다는 것을 보여줍니다.

실험 설정 MDLM과 같은 확산 모델은 양방향 어텐션을 사용하기 때문에 KV 캐싱을 지원하지 않습니다. BD3-LM과 같은 준-자기회귀 모델은 토큰 블록을 자기회귀적으로 생성하고 완료된 블록을 캐싱함으로써 부분적인 KV 캐싱을 가능하게 하지만, 확산 중인 현재 블록에 대한 캐싱은 여전히 부족합니다. 대조적으로, 우리 방법은 확산 단계 동안 KV 캐싱을 독점적으로 지원합니다. 우리는 우리의 방법인 Eso-LM (B)의 추론 시간을 MDLM 및 BD3-LM과 컨텍스트 길이 $L \in \{2048, 8192\}$으로 비교하며, first-hitting 샘플러 *

표 1: LM1B에서의 테스트 퍼플렉시티 (PPL; ↓). *He et al.

	PPL (↓)
자기회귀	
Transformer†	22.83
확산	
D3PM Uniform	

표 2: 25만 스텝 훈련 모델의 OWT 테스트 퍼플렉시티 (PPL; ↓). ‡

	PPL (↓)
자기회귀	
Transformer	17.90$
확산	
SEDD Absorb	

표 3: 모든 방법에서 NFEs를 L로 설정했을 때 시퀀스 길이 $L \in \{2048, 8192\}$에 대한 샘플링 시간 (↓, 초). 보고된 값은 5회 이상 실행의 평균입니다.

방법	**L = 2048**	**L = 8192**
AR (오라클)	13.30.9	54.00.2
MDLM	201.30.4	5438.33.3
BD3-LM (L' = 4)	24.30.7	312.01.7

방법	L = 2048	L = 8192
BD3-LM (L' = 16)	21.30.1	268.11.2
Eso-LM (B)	14.60.3	82.10.3

결과 표 3에 나타난 바와 같이, KV 캐싱이 없는 MDLM과 비교하여 Eso-LM (B)는

L=2048의 경우 ~14×배 더 빠르고, L=8192의 경우 ~65×배 더 빠릅니다. 부분적으로 캐싱을 지원하는 BD3-LM과 비교했을 때, Eso-LM (B)는 L=8192에서 BD3-LM (L′=16)보다 ~3.2×배 빠르며 BD3-LM (L′=4)보다 ~3.8×배 빠릅니다. 이러한 속도 향상은 KV 캐싱과, 노이즈 제거될 마스크 토큰 및 이전에 예측된 토큰으로 순방향 패스를 제한하여 중복 계산을 피하는 스케줄러 S에서 비롯됩니다. 이는 BD3-LM에는 없는 기능입니다. NFEs를 L로 제한하므로, 우리의 방법은 지연된 KV 재사용(마지막에서 두 번째 노이즈 제거 단계부터만 가능)으로 인해 AR 모델보다 느리지만, 전반적인 효율성은 여전히 이전 확산 기반 기준선을 능가합니다 (4.2.2절 참조).

5.3 생성 품질

우리의 샘플링 실험은 (1) Eso-LM이 샘플링 속도와 품질의 파레토 프론티어에서 새로운 SOTA를 확립하고 (그림 4), (2) 높은 NFEs에서 모든 이전 확산 모델보다 더 높은 품질의 샘플을 생성하며, (3) 이전 보간 확산 방법인 BD3-LM과는 달리 낮은 NFEs에서 모드 붕괴를 겪지 않음을 보여줍니다 (그림 3).

실험 설정 우리는 Eso-LM (B)를 α0train∈{0.125,0.25,1}로 훈련하고, (α0eval,T)∈{0.0625,0.25,0.5,1} ×{16,128,1024}을 다양하게 변경하여 샘플을 생성하여 NFEs를 제어합니다 (NFEs = |S|). MDLM과 BD3-LM은 Sahoo et al.

그림 3: OWT로 훈련된 모델의 생성 PPL 대 NFE. 엔트로피 값은 표 (8, 9, 10)에 있습니다.

그림 4: Eso-LM (B)는 샘플링 속도 및 품질의 파레토 프론티어에서 최신 기술(SOTA)을 확립합니다.

1 미만. 우리는 샘플 품질을 위해 생성 퍼플렉서티(GPT-2 Large를 통해)를 측정하고, 다양성을 위해 평균 엔트로피

생성 PPL 대 NFE의 파레토 프론티어 그림 3에서 보듯이, 높은 NFE(≈ 1024)에서 Eso-LM (B)는 MDLM 및 BD3-LM을 능가하며 AR 모델 품질과 거의 일치합니다. NFE가 감소함에 따라 BD3-LM은 교사 강제 학습으로 인해 크게 저하됩니다. 즉, 훈련 중에는 미래 블록이 깨끗한 이전 블록에 조건화되지만, 평가 중에는 이전 블록 예측의 오류가 샘플 품질의 상당한 저하로 이어집니다. Eso-LM은 이 문제를 피합니다. 예상대로, 낮은 NFE 영역에서는 더 높은 α0train (훈련 중 "더 많은 확산")가 품질을 향상시킵니다. α0train=1 는 α0train={0.125,0.25,1} 중에서 가장 좋은 성능을 보이며 MDLM과 경쟁력을 유지합니다. 전반적으로 Eso-LM (B)는 낮은 NFE에서는 MDLM과, 높은 NFE에서는 AR 모델과 경쟁력을 갖습니다(BD3-LM과는 다름).

생성 PPL 대 속도의 파레토 프론티어 샘플링 효율성을 비교하기 위해, 각 방법이 단일 샘플을 생성하는 데 걸린 중간 샘플링 시간(5회 시행에 걸쳐)을 초 단위로 기록하고, 그림 4에 생성 PPL 대 샘플링 시간을 플로팅합니다. 무한 시행의 한계에서, 중간 샘플링 시간은 NFE의 증가 함수이며, 사용된 방법과 샘플링 하이퍼파라미터에 의해 조절됩니다. 각 방법에 대해 모든 구성에 걸쳐 파레토 프론티어를 구성합니다: Eso-LM (B) (α0train , α0eval , T 에 걸쳐), BD3-LM (L′ 및 T 에 걸쳐), 그리고 MDLM (T 에 걸쳐)입니다. 우리는 Eso-LM (B)가 속도-품질 파레토 프론티어에서 새로운 최신 기술을 확립함을 발견했습니다. 샘플링 시간에 대한 예산이 적을 경우, α0train=1 으로 훈련된 모델이 가장 좋으며, 예산이 많을 경우, α0train=0.125 모델은 AR에 가까운 품질을 생성합니다. 중간 값(예: α0train={0.25,0.5})은 속도와 품질 사이에서 최상의 절충안을 제공합니다.

6 관련 연구, 논의 및 결론

AR 모델 AR 모델은 토큰을 왼쪽에서 오른쪽으로 생성하며 품질 면에서 최신 기술을 유지하지만, 느린 순차적 추론과 제한된 제어 가능성으로 인해 어려움을 겪습니다. 대조적으로, Eso-LM은 순차 단계에서의 AR 유사 생성과 초기 확산 단계에서의 임의 순서 병렬 생성을 결합합니다. 확산 중에 Eso-LM (B)는 KV 캐싱

마스크드 확산 MDM [26, 30]은 단계당 여러 토큰을 생성하지만, 비용이 많이 드는 양방향 어텐션에 의존합니다. Eso-LM은 두 가지 방식으로 효율성을 향상시킵니다. Eso-LM (A)는 어텐션을 깨끗하고 노이즈 제거가 예정된 토큰으로 제한하고, Eso-LM (B)는 KV 캐싱을 위해 AO-ARM

블록 확산 BD3-LM

훈련 중 교사 강제(teacher forcing) 사용으로 인해 후속 블록의 샘플 품질에 심각하게 영향을 미칩니다. Eso-LM은 이 문제로 고통받지 않습니다.

결론

우리는 자기회귀(AR) 모델과 마스크드 확산 모델(MDM)을 원활하게 융합하는 새로운 언어 모델링 패러다임을 소개하며, 샘플 품질과 생성 속도 모두에서 둘 사이의 효과적인 보간을 가능하게 합니다. 우리 방법은 병렬 생성을 유지하면서 MDM에서 KV 캐싱을 가능하게 한 최초의 방법으로, 추론을 크게 가속화합니다. 이 방법은 속도와 정확도 모두에서 블록 확산 방법을 능가하며, 언어 모델링 벤치마크에서 새로운 최신 기술을 수립합니다. 언어 모델링 연구를 수행하고 있다는 점을 감안할 때, 우리는 이 연구 분야에 내재된 위험과 기회를 안고 있습니다.

참고문헌

-

목차

1 서론 1 2 배경 2 2.1 자기회귀 모델 3 2.2 마스크 확산 모델 3 2.3 블록 노이즈 제거 확산 이산 언어 모델 3 3 난해한 언어 모델 4 3.1 자기회귀 모델과 마스크 확산의 융합 4 3.2 샘플링 5 4 공유 노이즈 제거 트랜스포머를 위한 어텐션 메커니즘 설계 5 4.1 Eso-LM (A) 6 4.2 Eso-LM (B) 8 5 실험 9 5.1 우도 평가 9 5.2 생성 속도 9 5.3 생성 품질 10 6 관련 연구, 논의 및 결론 11 부록 16 부록 A 배경 16 A.1 BD3-LMs 하이퍼파라미터 T′ 및 num_tries 16 부록 B 난해한 언어 모델 17 B.1 MDM 손실 17 B.2 훈련 절차 18 B.3 순차 단계의 어텐션 편향 18 B.4 샘플링 스케줄 및 샘플러 18 B.5 구현 세부 사항 19 B.6 실험 세부 사항 20 부록 C 추가 실험 22 C.1 분할 비율에 대한 제거 연구 22 C.2 OWT 혼란도 22 C.3 제로샷 우도 평가 22 C.4 OWT에서 훈련된 모델의 생성된 샘플의 생성 혼란도 및 엔트로피 23

부록

부록 A 배경

A.1 BD3-LMs 하이퍼파라미터 T′ 및 num_tries

BD3-LMs

또한, BD3-LMs의 코드베이스가 샘플을 선별한다는 것을 발견했습니다. 더 구체적으로 말해, 단일 샘플을 생성하기 위해 코드베이스는 하나의 샘플이 일부 품질 관리 테스트를 통과할 때까지 새로운 샘플을 계속 생성합니다(최대 num_tries 번). 기본적으로 num_tries = 10이며, 코드베이스는 10번의 시도가 모두 소진되어 테스트를 통과하는 샘플이 없을 경우 샘플링 실패를 보고합니다. 경험적으로, 우리는 T′=5000에 대해 샘플링 실패가 발생하지 않는다는 것을 발견했습니다.

병렬 생성에 대한 BD3-LMs의 실제 성능을 조사하기 위해 num_tries = 1로 설정하고 품질 관리 테스트를 비활성화한 다음, 넓은 범위의 T 값에 걸쳐 BD3-LMs의 샘플을 평가했습니다 (그림 5). 여기 그리고 그림 5에서 T은 BD3-LMs의 모든 블록에 걸친 샘플링 단계의 합계를 의미합니다. 예를 들어, L′=16 및 T=4096은 블록당 T′=4096/(1024/16)=64 샘플링 단계가 사용됨을 의미합니다. 반면, BD3-LMs의 코드베이스는 기본적으로 T′=5000를 사용하며, 이는 그림 5의 T=∞에 해당합니다. MDLM의 경우, T은 블록이 없으므로 정상적으로 해석될 수 있습니다.

그림 5에서 보듯이, 더 많은 병렬 생성을 가능하게 하기 위해 T이 감소함에 따라 BD3-LMs의 샘플 품질과 샘플 다양성 모두 MDLM보다 훨씬 나빠집니다. 이러한 성능 저하는 섹션 6에서 논의된 모드 붕괴 문제 때문입니다. 또한 num_tries를 늘리면 BD3-LMs의 샘플 엔트로피를 어느 정도 개선하고(표 4의 두 번째 행) 모드 붕괴를 완화할 수 있지만, AR 및 MDLM에는 개선 효과가 적거나 전혀 없다는 것을 발견했습니다.

!

표 4: BD3-LMs L′=16, MDLM 및 핵 샘플링 p=0.9을 사용한 AR의 생성 PPL (↓) 및 엔트로피 (↑) (괄호 안).

	BD3LM		**MDLM**		**AR**	
num_tries	1	10	1	10	1	10
T=1024	72.80 (5.35)	77.71 (5.41)	41.92 (5.36)	41.79 (5.37)	13.03 (5.26)	13.76 (5.32)
T=256	356.02 (5.11)	440.69 (5.28)	45.07 (5.40)	44.57 (5.39)	13.03 (5.26)	13.76 (5.32)

부록 B 난해한 언어 모델

B.1 MDM 손실

NLL은 다음과 같이 주어집니다:

−logpθ(x)≤−Ez0~q0(·|x)logpθAR(x|z0)+DKL(q0(z0|x) ‖ pθMDM(z0))=−Ez0~q0(·|x) [Σℓ∈M(z0)logpθAR(xℓ|z0,x<ℓ)]+DKL(q0(z0|x) ‖ pθMDM(z0)).₩tag39

z0은 인덱스 ℓ를 초과하는 인덱스에 클린 토큰을 포함할 수 있습니다. 섹션 3.1에서 논의했듯이 AR 로그-우도는 다음과 같이 주어집니다.

Σℓ∈M(z0)logpθAR(xℓ|z0,x<ℓ)=Σℓ∈M(z0)log(xθℓ(z0⦿x<ℓ),xℓ),₩tag40

여기서 우리는 마스크된 인덱스 $\mathcal{M}(\mathbf{z}0)$에서만 손실을 계산합니다.

(39)의 KL 항을 계산하기 위해, 우리는 $\mathbf{z}0$에 대한 마스크 확산 프로세스를 정의합니다. 이 확산 프로세스에 대해, 시간 t∈[0,1]에서의 전방 주변 분포 $\mathbf{z}t\ell \sim qt(\cdot|\mathbf{x}\ell)$는 (1)과 동일하지만, 축소된 범위 (αt)t∈[0,1]∈[0,α0]를 가진 노이즈 스케줄을 사용하며, 이는 t에 대해 엄격하게 감소하는 함수이고, αt=0=α0일 때 $\mathbf{z}t=0=\mathbf{z}0$이며, αt=1=0일 때 $\mathbf{z}t=1=\mathbf{m}1:L$입니다. T 확산 단계를 통해, 다음을 얻습니다:

DKL(q0(z0|x) ‖ pθMDM(z0))=Ez0[logq0(z0|x)pθMDM(z0)]=Ez0[logEz1T:1[q(z0:1|x)pθMDM(z0:1)]]≤Ez0:1[logq(z0:1|x)pθMDM(z0:1)]=Ez0:1[Σt∈{1T,2T,⋯,1}logq(zt−1T|zt,x)pθMDM(zt−1T|zt)]=ΣtEzt[DKL(q(zt−1T|zt,x) ‖ pθMDM(zt−1T

Sahoo et al.

=Et~U[0,1],zt~qt[αt′1−αtΣℓ∈M(zt)log(xθℓ(zt),xℓ)].₩tag41

마지막으로, (40)과 (41)을 결합하면 원하는 결과를 얻습니다:

LNELBO(x;θ)=Ez0~q0[−Σℓ∈M(z0)log(xθℓ(z0⦿x<ℓ),xℓ)⁓A R l o s s]
+∫t=0t=1αt′1−αtEzt~qt[Σℓ∈M(zt)log(xθℓ(zt),xℓ)]dt⁓M D M l o s s.₩tag42

B.2 훈련 절차

알고리즘 1은 완전한 훈련 절차를 간략하게 설명합니다.

알고리즘 1 Eso-LM 학습 입력: 데이터셋 D, 배치 크기 bs, 순방향 노이즈 프로세스 qt(·|$\mathbf{x}$), 모델 $\mathbf{x}$θ, 학습률 η converged 되지 않을 때까지 반복 $\mathbf{x}1,\mathbf{x}2,\cdots,\mathbf{x}$bs~D i←1부터 bs/2까지 반복 ▷ 만약 α0=1이면, 1부터 bs까지 반복. $\mathbf{z}0$~q0(·|$\mathbf{x}$) σ~$\mathcal{P}$L 제약 조건을 사용하며, $\mathbf{x}$θ (섹션 4)에서 어텐션 편향 A을 구성하는 데 사용됨 $\mathcal{A}$←−Σℓ∈$\mathcal{M}$($\mathbf{z}$0)log($\mathbf{x}$θℓ($\mathbf{z}$0,$\mathbf{x}$<ℓ),$\mathbf{x}$iℓ) (7)의 순차 손실 추정기 반복 종료 i←bs/2+1부터 bs까지 반복 ▷ 만약 α0=1이면, 이 반복을 건너뜀. t~$\mathcal{U}$[0,1] 샘플링 $\mathbf{z}$t~qt(·|$\mathbf{x}$) σ~$\mathcal{P}$L 제약 조건을 사용하며, $\mathbf{x}$θ (섹션 4)에서 어텐션 편향 A을 구성하는 데 사용됨 $\mathcal{A}$←αi′1−αiΣℓ∈$\mathcal{M}$($\mathbf{z}$t)log($\mathbf{x}$θℓ($\mathbf{z}$t),$\mathbf{x}$iℓ) (7)의 MDM 손실 추정기 반복 종료 θ←θ−η▽θΣi=1bs$\mathcal{A}$ 반복 종료

B.3 순차 단계에 대한 어텐션 편향

그림 6은 순차 단계 학습에 대한 어텐션 편향 비교를 보여줍니다.

!

B.4 샘플링 스케줄 및 샘플러

Eso-LM은 확산 단계와 순차 단계의 두 가지 샘플링 단계를 수행합니다. 확산 단계에서는 토큰이 무작위 순서로, 잠재적으로 병렬로 노이즈 제거됩니다. 순차 단계에서는 남아있는 마스크 토큰이 왼쪽에서 오른쪽으로 하나씩 순차적으로 노이즈 제거됩니다.

먼저, (i) 확산 단계 동안 노이즈 제거할 전체 토큰 수와 (ii) 확산 단계당 노이즈 제거할 토큰 수를 결정하기 위해,

---

T 단계. 각 단계에 대해, 시간 t에 노이즈 제거할 토큰 수를 다음과 같이 계산합니다.

nt=Binom(n=ntremaining,p=αs−αt1−αt), (43)

여기서 s=t−dt 및 ntremaining=L−Σt′>tnt′입니다. T가 클 때, 일부 nt는 0이 될 수 있습니다. 이 알고리즘에 의해 생성된 모든 nt는 0인 nt를 제외하고 정렬된 목록에 수집됩니다. 모든 nt의 합을 nMDM로 표시하고 nAR=L−nMDM을 정의합니다.

우리는 [L]에서 nMDM개의 토큰 인덱스를 선택하여 확산 방식으로 노이즈 제거하고, 보완적인 토큰 인덱스 부분집합을 사용하여 순차적으로 노이즈 제거합니다. 예를 들어, L=8이고 토큰 인덱스가 [1,2,⋯,8]이라고 가정합니다. 위 알고리즘에서 nMDM=5를 얻었다고 가정합니다. 그러면 우리가 선택할 수 있는 확산 인덱스는 (1,3,4,6,7)이고 보완적인 순차 인덱스는 (2,5,8)입니다. 우리는 확산 인덱스를 무작위 순서 노이즈 제거를 위해 (3,1,6,4,7)과 같이 무작위로 재배열합니다.

0이 아닌 nt들의 목록과 재배열된 확산 인덱스의 정렬된 집합이 주어지면, nt에 따라 확산 인덱스를 분할하여 확산 샘플링 스케줄을 만듭니다. 0이 아닌 nt들의 목록이 (2,1,2)이라고 가정합니다. 이를 사용하여 재배열된 확산 인덱스 집합 (3,1,6,4,7)를 분할하면 확산 단계에 대한 다음 샘플링 스케줄 $\mathcal{S}$MDM=((3,1),(6),(4,7))를 얻습니다. 순차 단계에 대한 노이즈 제거 스케줄은 단순히 $\mathcal{S}$AR=((2),(5),(8))입니다. 통합 샘플링 스케줄 $\mathcal{S}$은 $\mathcal{S}$MDM와 $\mathcal{S}$AR의 연결입니다. 이 예에서, $\mathcal{S}$=(S1,S2,S3,S4,S5,S6)이고 여기서 S1=(3,1),S2=(6),S3=(4,7),S4=(2),S5=(5) 및 S6=(8)입니다. 이는 6개의 NFE에 해당합니다. 마지막으로, $\mathcal{S}$은 알고리즘 2로 전달되어 나머지 샘플링 절차를 처리합니다. 섹션 4.1.2 및 섹션 4.2.2에서 논의된 노이즈 제거 순서 σ와 다시 연결하면, 이 예에서는 σ=(3,1,6,4,7,2,5,8)입니다.

입력: 시퀀스 길이 L, 통합 샘플링 스케줄 $\mathcal{S}$ $\mathbf{z}$=[MASK_INDEX,⋯,MASK_INDEX] C={} ▷ 클린 토큰의 인덱스 i←1부터 |$\mathcal{S}$|까지 반복 ▷ |C|≥nMDM일 때 순차적으로 자동 발생 logits ←$\mathbf{x}_\theta$($\mathbf{z}$[C∪Si]) ▷ 비고 참조 logits ← Si에 해당하는 logits 선택 $\mathbf{z}$[Si]←categorical_sample(logits, dim=-1) ▷ logits의 모양은 (|Si|,|$\mathcal{V}$|) C←C∪Si 반복 종료 반환: $\mathbf{z}$ 비고. $\mathbf{z}$[C∪Si]은 노이즈 제거 모델 $\mathbf{x}_\theta$에 입력되는 $\mathbf{z}$ 내 토큰의 부분집합을 나타냅니다. 토큰 $\mathbf{z}_\ell \in \mathbf{z}$[C∪Si]의 위치 임베딩은 원래 시퀀스 $\mathbf{z}$에서의 위치 임베딩과 동일하게 유지됩니다. Eso-LM (A) 및 Eso-LM (B)에 대한 샘플링 어텐션 편향 A 계산은 각각 섹션 4.1.2 및 섹션 4.2.2를 참조하십시오. Eso-LM (B)의 경우, 인과적 어텐션 사용으로 인해, $\mathbf{x}_\theta$는 클린 토큰이 처음 처리될 때 해당 KV-값을 캐시할 수 있습니다.

알고리즘 2 Eso-LM 샘플링

B.5 구현 세부 사항

B.5.1 학습

확산 단계

Eso-LM (A)와 Eso-LM (B) 모두에서, 우리는 입력 $\mathbf{z}_t$와 A의 행과 열을 σ에 따라 정렬합니다. 이는 효율적으로 구현될 수 있는 고전적인 어텐션 편향(그림 7)을 만듭니다. 또한 토큰이 원래 위치 임베딩을 유지하도록 $\mathbf{z}_t$의 위치 임베딩을 σ에 따라 정렬합니다. 손실을 계산할 때, 우리는 추가적으로 목표 $\mathbf{x}$를 σ에 따라 정렬합니다.

순차 단계

Eso-LM (A)와 Eso-LM (B) 모두에서, 우리는 $\mathbf{z}_0$, $\mathbf{x}$ (입력 $\mathbf{z}_0 \oplus \mathbf{x}$로 연결하기 전), 그리고 A 내의 네 L×L 블록 각각의 행과 열을 σ에 따라 정렬합니다. 우리는 $\mathbf{z}_0$의 클린 토큰에 대해 특정 추가 어텐션 연결을 추가합니다. 이는 $\mathbf{z}_0$에서 다른 토큰이 클린 토큰에 어텐션하지 않기 때문에 트랜스포머 출력에 영향을 미치지 않습니다. 이 두 가지 변환은 효율적으로 구현될 수 있는 희소하고 패턴화된 블록 구조(그림 8)를 가진 어텐션 편향을 만듭니다. 또한 토큰이 원래 위치 임베딩을 유지하도록 $\mathbf{z}_0$와 $\mathbf{x}$의 위치 임베딩을 σ에 따라 정렬합니다. 손실을 계산할 때, 우리는 추가적으로 목표 $\mathbf{x}$를 σ에 따라 정렬합니다.

!

!

B.5.2 샘플링

그림 9는 Eso-LM (A)의 샘플링 절차가 예시 시퀀스를 생성하는 방법을 보여줍니다. 그림 10은 Eso-LM (B)의 샘플링 절차가 예시 시퀀스를 생성하는 방법을 보여줍니다.

B.6 실험 세부 사항

B.6.1 저편차 샘플러

학습 중 분산을 줄이기 위해 Kingma et al.

B.6.2 우도 평가

우리는 우도를 평가하기 위해 t에 대한 단일 몬테카를로 추정치를 사용합니다. 추정치의 분산을 줄이기 위해 저편차 샘플러

!

!

B.6.3 언어 모델링

[15, 26]의 코드를 따라 One Billion Words 데이터셋을 토큰화 해제합니다. Austin et al.

OpenWebText를 GPT2 토크나이저로 토큰화합니다. 이들을 연결하고 1,024 길이로 묶습니다. 묶을 때, 연결된 시퀀스들 사이에 eos 토큰을 추가합니다. One Billion Words와 달리, OpenWebText의 최종 전처리된 시퀀스는 첫 번째 및 마지막 토큰으로 특수 토큰을 가지지 않습니다. OpenWebText는 검증 분할이 없으므로, 마지막 10만 개의 문서를 검증용으로 남겨둡니다.

Eso-LM은 Lou et al.

우리는 로그 선형 노이즈 스케줄 $\alpha t = \alpha 0(1-t)$ 를 사용합니다. 우리는 배치 크기 512의 AdamW 옵티마이저를 사용하며, 2,500 스텝 동안 학습률 0에서 3e-4까지 일정하게 학습률 웜업을 적용합니다. 우리는 One Billion Words에 대해 1백만 스텝, OpenWebText에 대해 25만 스텝 동안 일정한 학습률을 사용합니다. 우리는 0.1의 드롭아웃 비율을 사용합니다. 우리는 H200 GPU에서 모델을 훈련시킵니다. One Billion Words에서 1백만 스텝 동안 훈련은 $\alpha 0=1$일 때 ~21시간이 걸리고, $\alpha 0<1$일 때 ~34시간이 걸립니다. OpenWebText에서 25만 스텝 동안 훈련은 $\alpha 0=1$일 때 ~50시간이 걸리고, $\alpha 0<1$일 때 ~69시간이 걸립니다.

부록 C 추가 실험

C.1 분할 비율에 대한 절제 연구

표 5를 참조하십시오.

표 5: LM1B에서 500K 스텝 동안 훈련된 Eso-LM (A)에 대한 테스트 퍼플렉시티 (↓)와 훈련 중 (7)의 MDM 손실 평가에 사용된 각 배치 내 예시 비율. 각 배치 내 나머지 예시들은 훈련 중 (7)의 AR 손실 평가에 사용됩니다.

	0.75	**0.5**	**0.25**	**0.125**
Eso-LM (A)				
α0=0.5	32.25	31.53	발산함	발산함
α0=0.25	30.49	29.33	발산함	발산함
α0=0.125	27.76	26.73	발산함	발산함
α0=0.0625	25.92	25.07	발산함	발산함

C.2 OWT 퍼플렉시티

표 6을 참조하십시오.

표 6: OWT에서 25만 스텝 및 1백만 스텝 동안 훈련된 모델에 대한 테스트 퍼플렉시티(PPL; ↓). 우리는 확산 모델 및 보간법에 대한 상한/하한을 보고합니다. ǂ

	PPL 250K (↓)	**PPL 1M (↓)**
자기회귀		
Transformer	17.90|17.54	
확산		
SEDD Absorb		

C.3 제로샷 우도 평가

우리는 또한 OWT에서 훈련된 모델을 사용하여 보이지 않는 데이터셋을 얼마나 잘 모델링하는지 평가함으로써 모델의 일반화 능력을 탐구합니다 (표 7). 우리는 우리의 Eso-LM의 퍼플렉시티를 SEDD

표 7: OWT에서 25만 스텝 동안 훈련된 모델의 제로샷 퍼플렉시티 (↓). 우리는 확산 모델 및 보간법에 대한 상한/하한을 보고합니다. AR에 대한 수치는

	PTB	**Wikitext**	**LM1B**	**Lambada**	**AG News**	**Pubmed**	**Arxiv**
AR	81.07	25.32	51.14	52.13	52.11	48.59	41.22
MDLM	93.82	36.89	69.45	53.05	67.33	49.47	43.84
SEDD Absorb	99.59	38.55	72.51	52.16	72.62	47.07	41.18
BD3-LM (L' = 16)	90.63	33.14	64.88	53.09	62.5	43.25	39.82
Eso-LM (B) (본 연구)							
α0 = 1	126.29	45.08	82.01	61.37	98.22	62.37	55.76
α0 = 0.5	110.70	39.57	75.75	57.33	86.65	60.20	53.78
α0 = 0.25	105.19	37.32	67.69	60.15	75.74	62.45	55.31
α0 = 0.125	97.46	35.65	60.11	69.13	65.26	65.27	57.4

C.4 OWT에서 훈련된 모델이 생성한 샘플의 생성 퍼플렉시티 및 엔트로피

그림 3에서 우리는 NFE를 변화시킴으로써 샘플 품질이 어떻게 변하는지 연구합니다. 생성 PPL 및 엔트로피의 수치는 표 8 (Eso-LM), 표 9 (MDLM), 표 10 (BD3-LM)에서 확인할 수 있습니다.

표 8: OWT에서 25만 스텝 동안 훈련된 Eso-LM (B)의 샘플에 대한 생성 PPL (↓) 및 엔트로피 (↑).

α0train	**α0eval**	**T**	**NFE**	**생성 PPL (↓)**	**엔트로피 (↑)**
0.125	0.0625	16	977.18	23.15	5.37
0.125	0.0625	128	1011.37	25.51	5.41
0.125	0.0625	1024	1022.13	26.85	5.43
0.125	0.25	16	806.47	46.68	5.45
0.125	0.25	128	896.41	44.19	5.42
0.125	0.25	1024	999.24	49.55	5.47
0.125	1.0	16	16.0	279.63	5.54
0.125	1.0	128	127.96	151.77	5.4
0.125	1.0	1024	647.06	159.31	5.42
0.25	0.0625	16	977.18	23.95	5.36
0.25	0.0625	128	1011.37	26.13	5.39
0.25	0.0625	1024	1022.13	27.96	5.41
0.25	0.25	16	806.47	42.31	5.43
0.25	0.25	128	896.41	38.48	5.39
0.25	0.25	1024	999.24	43.71	5.44
0.25	1.0	16	16.0	152.16	5.55
0.25	1.0	128	127.96	72.74	5.4
0.25	1.0	1024	647.06	72.36	5.42
1.0	0.0625	16	977.18	31.33	5.21
1.0	0.0625	128	1011.37	30.9	5.19
1.0	0.0625	1024	1022.13	31.11	5.19
1.0	0.25	16	806.47	50.14	5.4
1.0	0.25	128	896.41	43.59	5.33
1.0	0.25	1024	999.24	43.6	5.32
1.0	1.0	16	16.0	118.92	5.53
1.0	1.0	128	127.96	54.36	5.37
1.0	1.0	1024	647.06	49.4	5.33

표 9: OWT에서 1백만 스텝 동안 훈련된 MDLM의 샘플에 대한 생성 PPL (↓) 및 엔트로피 (↑).

T	NFE	생성 PPL (↓)	엔트로피 (↑)
16	16	148.08	5.61
128	127.96	48.66	5.43
256	251.35	45.07	5.4
512	442.87	42.58	5.37
1024	647.91	41.92	5.36
4096	906.32	41.53	5.36

표 10: OWT에서 1백만 스텝 동안 훈련된 BD3-LM의 샘플에 대한 생성 PPL (↓) 및 엔트로피 (↑).

블록 크기	T	NFE	생성 PPL (↓)	엔트로피 (↑)
4	4096	925.32	41.04	5.31
4	2048	830.93	88.4	5.19
4	1024	672.91	199.99	4.79
4	512	480.21	186.05	4.23
4	256	256.0	199.22	4.39
8	4096	915.14	39.88	5.33
8	2048	814.07	59.84	5.34
8	1024	646.69	137.99	5.24
8	512	439.32	258.95	4.92
8	256	254.9	278.57	4.39
8	128	128	466.1	4.91
16	4096	910.37	38.74	5.33
16	2048	811.01	47.02	5.34
16	1024	642.25	73.84	5.36
16	512	427.46	189.26	5.33
16	256	245.77	351.63	5.1
16	128	128	447.24	4.61